

Modelado de datos

Con la finalidad de cubrir las necesidades del sistema médico y de las conversaciones con el stakeholder, se plantea un modelo estrella para mejorar la gestión de la información disponible. Lo anterior define bien los datos requeridos para las métricas a evaluar por parte del cliente. La figura 1 muestra en modelo de datos de tipo estrella que se plantea para el presente proyecto.

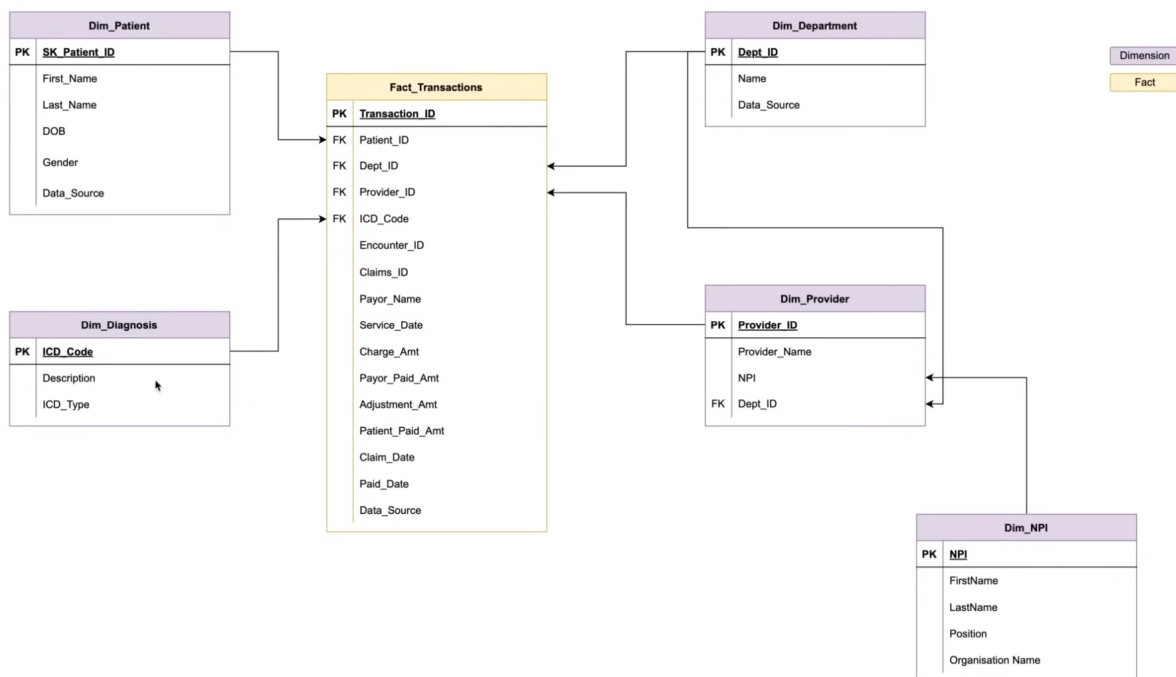


Figura 1.- Modelo de datos tipo estrella para el proyecto Health-care de ingeniería de datos en Azure.

Descripción de actividades

Durante la ingesta de datos se tienen 4 fuentes principales, como 2 bases de datos de Azure simulando un sistema EMR (una para el hospital-A y otra para el hospital-B) con información de las transacciones, pacientes, proveedores, citas y departamentos.

Archivos planos de tipo csv, simulando la entrega manual del stakeholder los cuales se almacenan en la capa landing (archivos claim y cpt-codes).

La extracción de información de 2 API's públicas para los datos de los códigos NPI e ICD.

Para la ingesta se utiliza un pipeline para depositar los datos en la capa bronze en formato parquet. Se utiliza Azure Data Factory y se aplica una ingesta tanto tipo Full e Incremental dependiendo de los datos. Con databricks convertimos los archivos csv de la carpeta landing a la bronze así como también se extrae la información de las API's y se copian como archivo parquet en la capa bronze.

Posteriormente y en función de nuestro modelo de datos, se aplica un segundo pipeline para mover, y transformar los datos de las capas bronze a silver y finalmente a gold para completar la arquitectura medallón, lo anterior en formato delta lake y empleando el unity catalog en databricks. Se aplicaron técnicas como la creación de un Common Data Model para empatar los schemas de las tablas, la validación de la calidad del dato para enviar a cuarentena datos inválidos y un Slowly Changing Dimension Type-2, para preservar información histórica y actualizar los cambios en los datos.

Como resultado se obtienen los componentes del modelos de datos dentro del unity catalog y en el data lake de Azure, es decir las fact y dimension tables actualizadas y listas para consultas alineadas a los insights de interés. El proyecto me permitió aplicar conceptos como el manejo de infraestructura y/o servicios de la nube en Azure, uso de un data lake para el almacenamiento, aplicación de Data Factory para la orquestación del flujo de trabajo, manejo de la plataforma lakehouse de databricks, su capa de almacenamiento delta lake así como el gobierno de datos con el unity catalog, aplicación de principios de modelado de datos con el modelo estrella para la gestión de la información y de la limpieza y calidad del dato con SCD-2 y el manejo de distintos tipos de archivos como csv's, parquet y delta (figura 2).

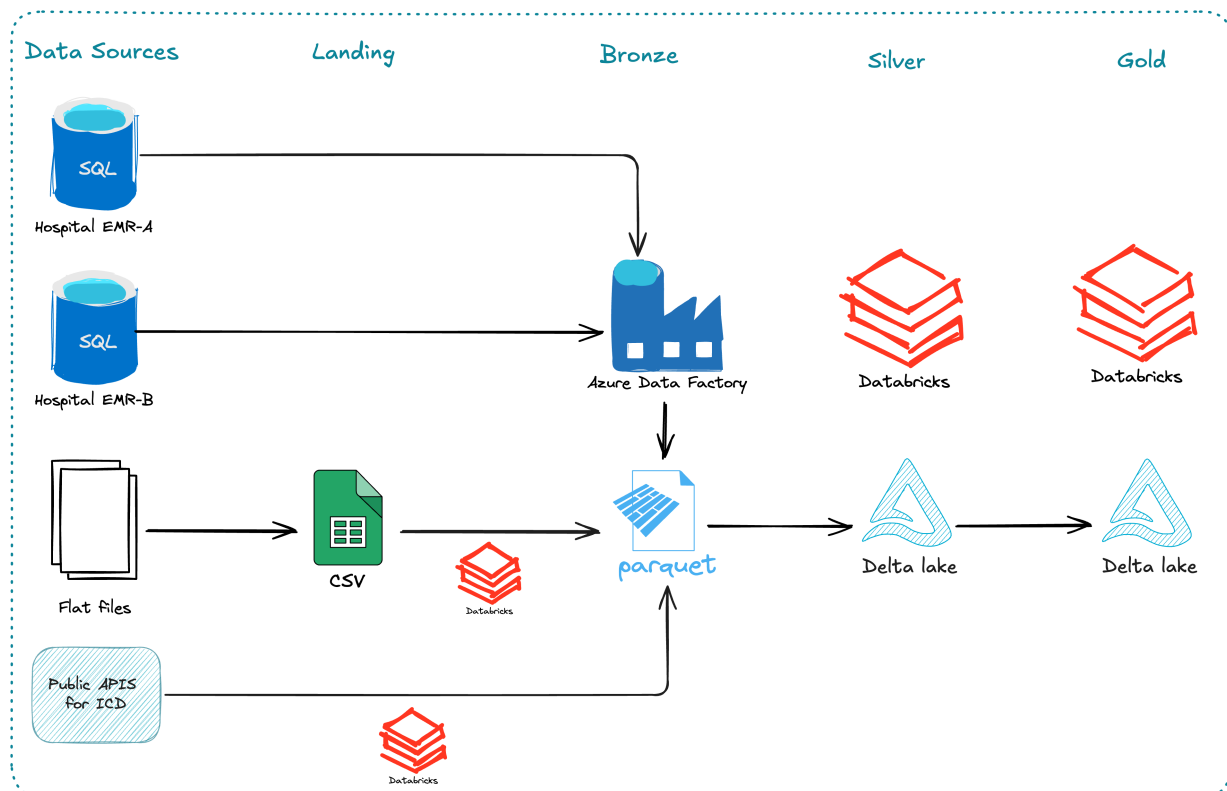


Figura 2.- Arquitectura medallón de datos para el proyecto health-care en Azure.

Pipeline para cargar los datos a las bases de Azure-SQL Database

Para simular el almacenamiento de los datos en el EMR se desplegaron 2 bases de datos en Azure SQL, así como también un respectivo servidor. Una vez que se se tienen las bases, se crearon las tablas para cada hospital empleando el código del Data Definition Language – DDL, el cual viene en la carpeta EMR_DDL. A manera de ejemplo se muestra el código de las 3 primeras tablas para el hospital-A.

-- For database trendytech-hospital-a:

-- 1. Departments Table:

```
CREATE TABLE dbo.departments (  
    DeptID nvarchar(50) NOT NULL,  
    Name nvarchar(50) NOT NULL,  
    CONSTRAINT PK_departments PRIMARY KEY (DeptID)  
);
```

-- 2. Encounters Table:

```
CREATE TABLE dbo.encounters (  
    EncounterID nvarchar(50) NOT NULL,  
    PatientID nvarchar(50) NOT NULL,  
    EncounterDate date NOT NULL,  
    EncounterType nvarchar(50) NOT NULL,  
    ProviderID nvarchar(50) NOT NULL,  
    DepartmentID nvarchar(50) NOT NULL,  
    ProcedureCode int NOT NULL,  
    InsertedDate date NOT NULL,  
    ModifiedDate date NOT NULL,  
    CONSTRAINT PK_encounters PRIMARY KEY (EncounterID)  
);
```

-- 3. Hospital1_Patient_Data Table:

```
CREATE TABLE dbo.patients (  
    PatientID nvarchar(50) NOT NULL,  
    FirstName nvarchar(50) NOT NULL,  
    LastName nvarchar(50) NOT NULL,  
    MiddleName nvarchar(50) NOT NULL,  
    SSN nvarchar(50) NOT NULL,  
    PhoneNumber nvarchar(50) NOT NULL,  
    Gender nvarchar(50) NOT NULL,  
    DOB date NOT NULL,  
    Address nvarchar(100) NOT NULL,  
    ModifiedDate date NOT NULL,  
    CONSTRAINT PK_hospital1_patient_data PRIMARY KEY (PatientID)  
);
```

Una vez que creadas las tablas, procedemos a poblarlas. Para esto podemos crear un Metadata-driven pipeline dentro de Azure Data Factory, el cual copiará desde archivos csv a cada base de datos. En el datalake creamos una carpeta llamada “raw-data-for-sql-database” en la cual se almacenan los archivos para cada hospital y un archivo .json con la metadata para el pipeline (figura 3).

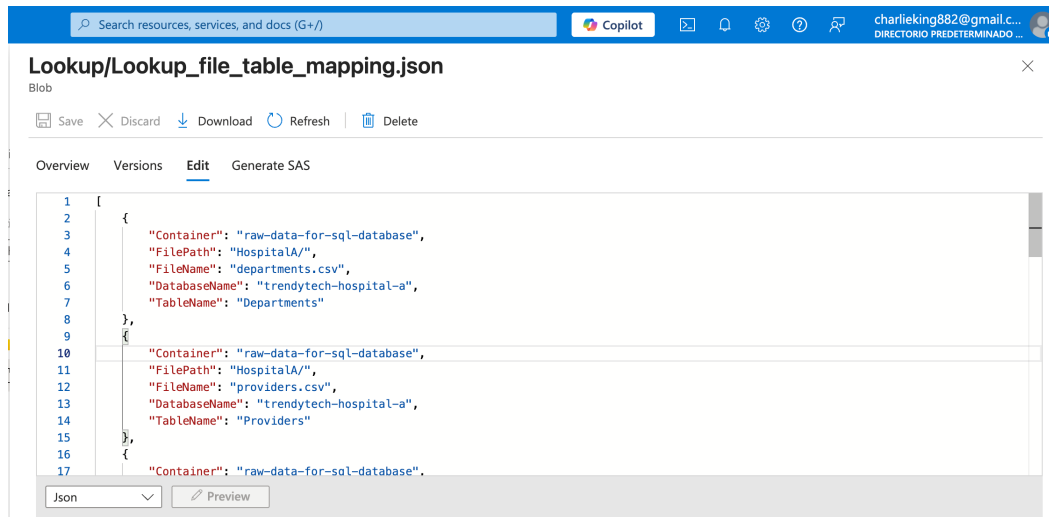


Figura 3.- Archivo json con la metadata para el pipeline dinámico de carga de datos hacia azure sql database.

Posteriormente procedemos a construir el pipeline, el cual consiste en una actividad “Lookup” para la lectura del archivo json y un bucle for para iterar en cada archivo o item del archivo json. Dentro del for loop, se tiene una actividad de copiado de datos, en el cual la fuente o source es el data lake y el destino o sink es son las bases de datos en Azure SQL.

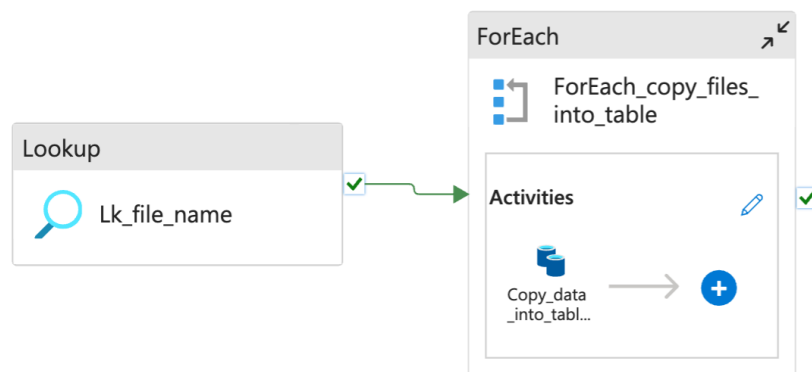


Figura 4.- Pipeline para la carga de datos en las bases de Azure-SQL.

En la copy activity dentro del for loop se defnieron de manera dinámica los parametros para el nombre de archivo, ruta y contenedor como se muestra en la figura 5.

pl_to_insert_data_to_sql_table_preprocessing > ForEach_copy_files_into_table

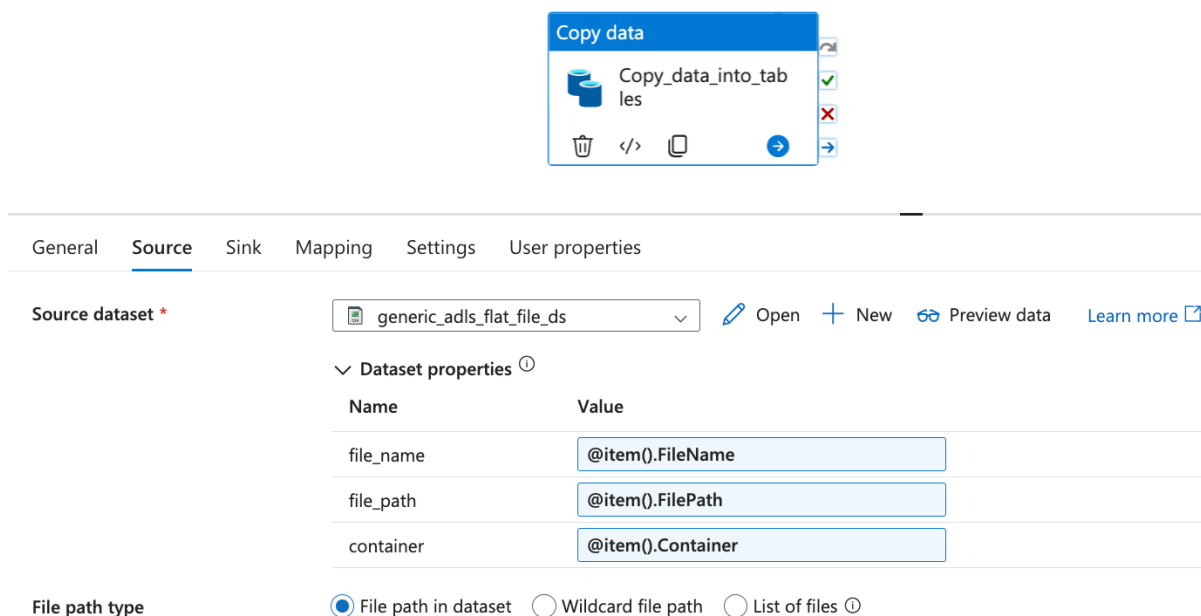


Figura 5.- Definición de parametros como el nombre de archivo, ruta y contenedor para el copiado en las bases de datoa de Azure.

Una vez que corremos nuestro pipeline, podemos verificar la correcta carga de los datos en cada base, en el query editor(preview). Las figuras 6 y 7 muestran los datos para la base del hospital-A y hospital-B.

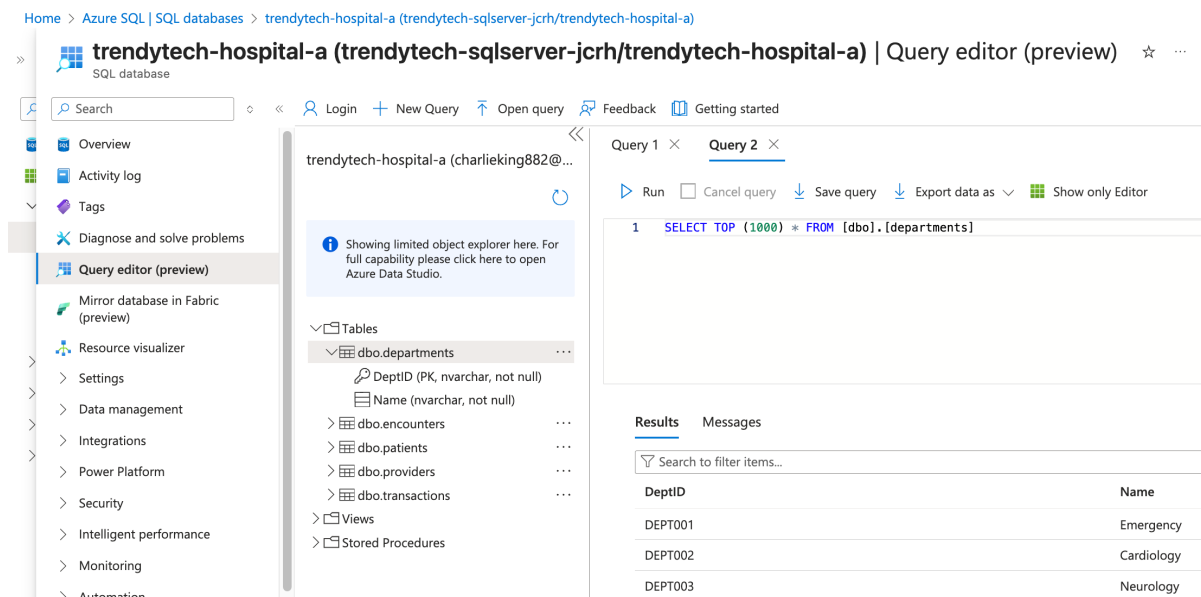


Figura 6.- Verificación de la correcta carga de datos en la base Azure (EMR) para el Hospital-A. Se observan las 5 tablas como departamentos, encounters, patients, providers y transactions. A su vez se muestran los primeros valores de la tabla departamentos.

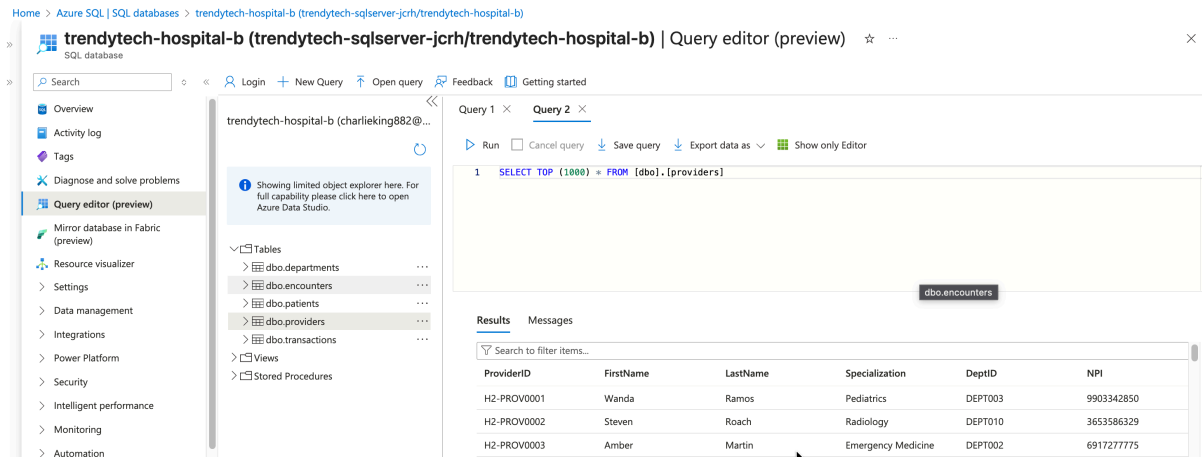


Figura 7.- Verificación de la correcta carga de datos en la base Azure (EMR) para el Hospital-B.

Pipeline para mover datos desde las bases EMR de Azure SQL DB hacia la capa Bronce

- La actividad Lookup lee el archivo *config_load.csv* ubicado en la carpeta configs del data lake.
- Inicia un foreach activity que recorrerá cada una de las filas del archivo “config_load.csv”.
- Se verifica si el archivo ya existe en la carpeta bronze del data lake en caso de cumplirse la condición el archivo se copia a la carpeta “**archive**” con la fecha de carga del año, mes y día, lo anterior debido a que los archivos en la carpeta bronze se van a actualizar de manera “**Full**” o “**Incremental**”.
- Se define una segunda condición If para evaluar si se hará una carga “Full” o “Incremental” de los datos, dependiendo de como se tenga definido en el archivo load_config.csv.

Para una carga tipo Full, tenemos:

- En la actividad Copy Activity se copian los archivos de las bases de datos de Azure SQL Hospital-A y Hospital-B y se agregan columnas como la de datasource y la de tablename para una mejor trazabilidad de los datos.
- Se carga en la tabla delta audit.load_logs en el schema audit de databricks con las columnas siguientes:

data_source	tablename	Numberofrowscopied	watermarkcolumnname	loaddate

Para una carga tipo Incremental, tenemos:

- En la actividad Lookup llamada “Fetch-logs”, se extrae a través de una query SQL a la tabla audit_table de databricks, la ultima fecha(loaddate) de actualización, para filtrar los datos nuevos.
- En la actividad Copy Activity se copian los archivos de las bases de datos de Azure SQL Hospital-A y Hospital-B. Se ejecuta una query para filtrar los datos posteriores a la última fecha de carga(loaddate).
- Se carga en la tabla delta audit.load_logs en el schema audit de databricks.

El diagrama del pipeline completo se observa en la figura siguiente:

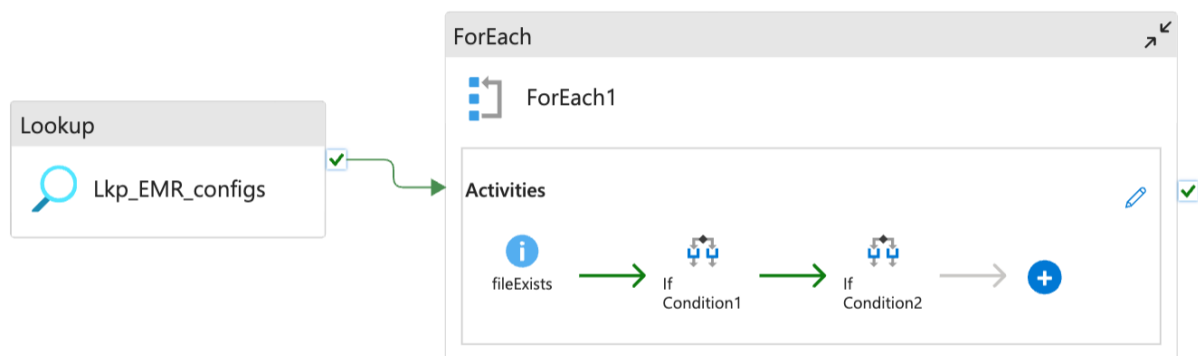


Figura 8.- Flujo de datos para la carga de las bases de datos Azure SQL a la capa bronce.

En la figura 2 podemos observar mayor detalle dentro del bucle ForEach, el cual contiene el Get Metadata así como las dos condicionales If.

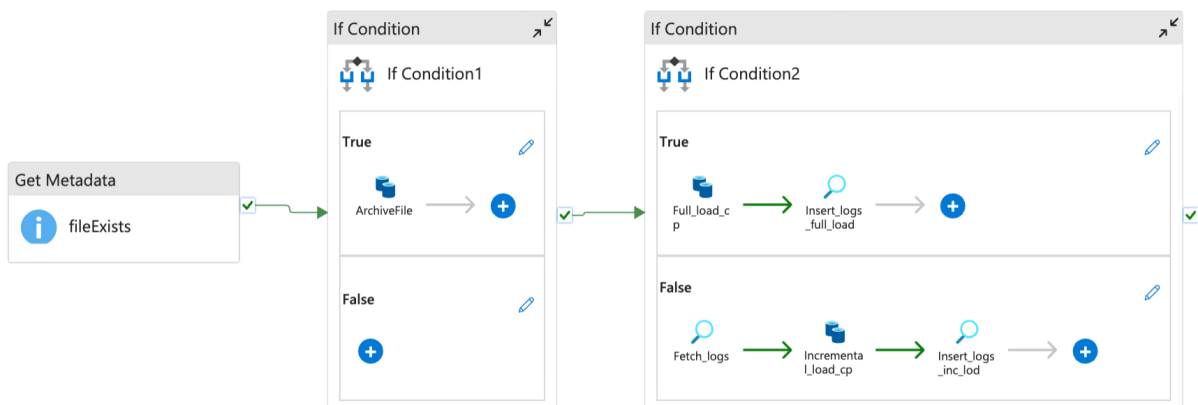


Figura 9.- Detalle del flujo y actividades a ejecutar dentro del ForEach para las cargas “Full” e “Incremental”.

En la figura 3 y 4 se muestran las actividades para una carga de tipo Full y una de tipo Incremental, las cuales consisten en actividades tipo Copy y Lookup.



Figura 10.- Flujo dentro del segundo if-condition para una carga de datos de tipo “Full”.

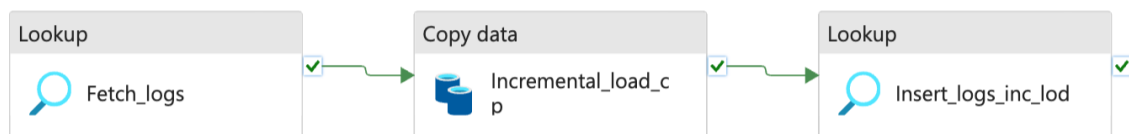


Figura 11.- Flujo dentro del segundo if-condition para una carga de datos de tipo “Incremental”.

PRIMERA CORRIDA.- CARGA DE DATOS CON DATALAKE VACIO.

Se parte desde cero, considerando que el datalake no tiene ningun archivo en la capa bronze, como se puede observar en las figura 5 y 6.

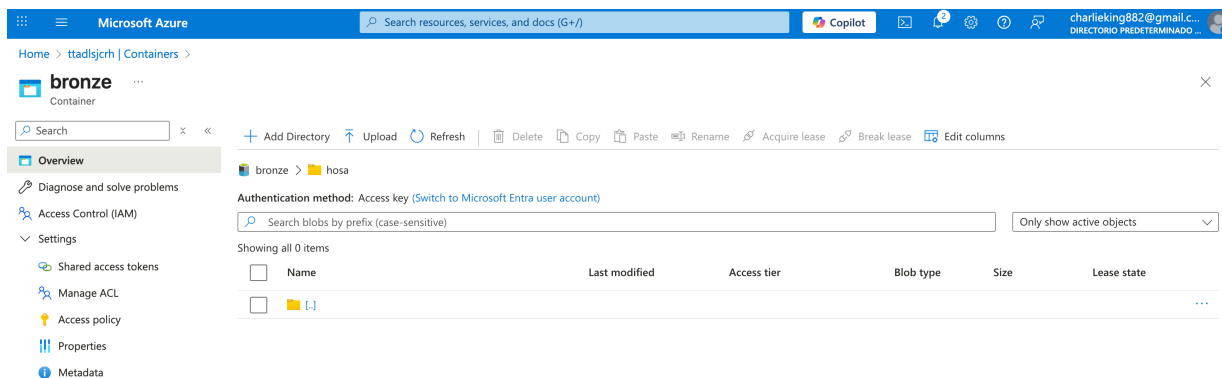


Figura 12.- Capa bronze dentro del data lake donde se obverva vacío para los archivos del hospital-A.

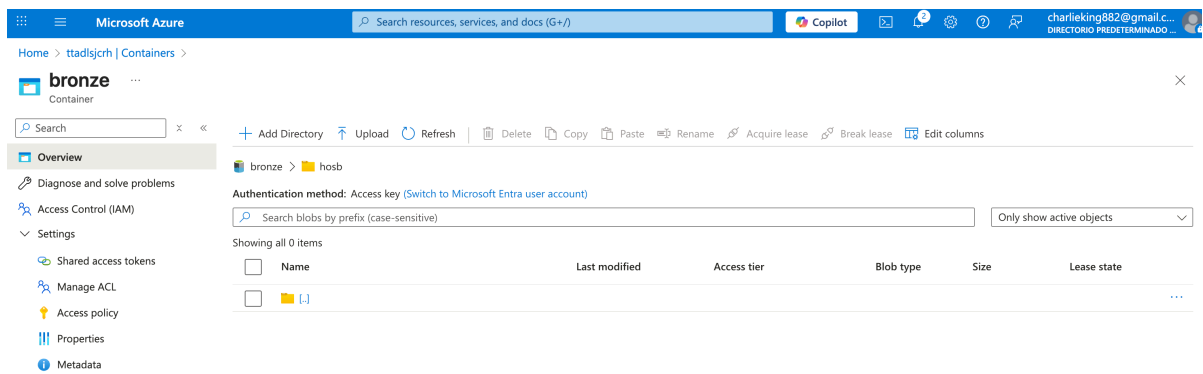


Figura 13.- Capa bronce dentro del data lake donde se observa vacío para los archivos del hospital-B.

Posterior a la carga de datos y conversión a archivos parquet podemos verificar en el contenedor del data lake que se han generado de manera correcta, como se observa en las figuras 7 y 8.

Por último, verificamos la actualización de la tabla delta en databricks, en la cual podemos notar 10 registros, 1 por fila dentro del archivo config.

id	data_source	tablename	numberofrowscopied	watermarkcolumnname	loaddate
1	hos-a	dbo.encounters	10000	ModifiedDate	2025-12-25T23:16:23.32
2	hos-a	dbo.patients	5000	ModifiedDate	2025-12-25T23:17:33.71
3	hos-a	dbo.transactions	10000	ModifiedDate	2025-12-25T23:18:52.49
4	hos-a	dbo.providers	25		2025-12-25T23:19:57.10
5	hos-a	dbo.departments	20		2025-12-25T23:20:55.32
6	hos-b	dbo.encounters	10000	ModifiedDate	2025-12-25T23:22:26.95
7	hos-b	dbo.patients	5000	Updated_Date	2025-12-25T23:23:51.95
8	hos-b	dbo.transactions	10000	ModifiedDate	2025-12-25T23:25:29.02
9	hos-b	dbo.providers	30		2025-12-25T23:26:01.43
10	hos-b	dbo.departments	20		2025-12-25T23:26:50.75

Tabla 1.- Audit table para la posterior a la primera corrida del pipeline.

SEGUNDA CORRIDA.- CARGA DE DATOS CON DATA LAKE LLENO.

En este segundo ejercicio en el data lake ya se tienen los archivos parquet cargados previamente de la primera corrida, simulando que ya existieran los mismos. Estos archivos deben copiarse en la carpeta “archive” para su historial, como lo marca el pipeline.

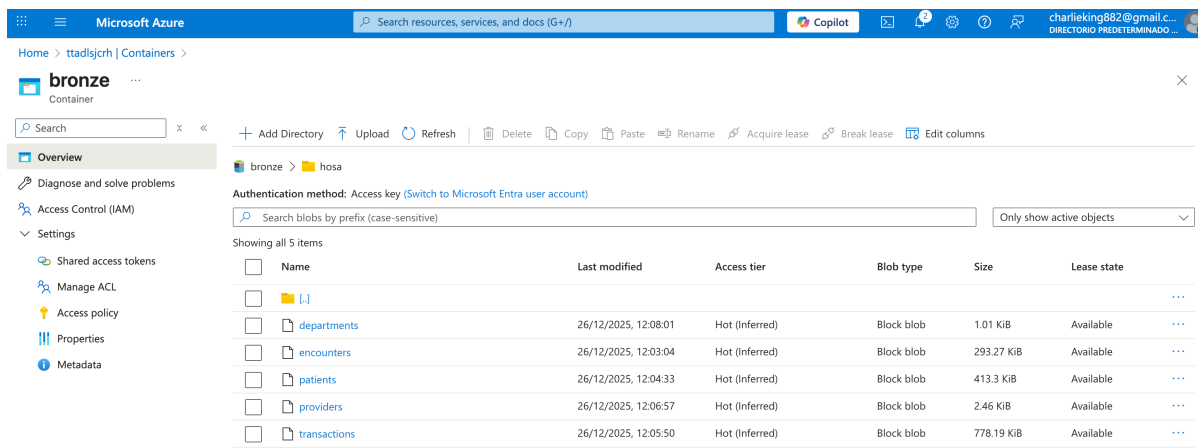


Figura 14.- Capa bronce dentro del data lake donde se observa la carga de los archivos parquet del hospital-A.

Posterior a la corrida notamos que se agregó la carpeta archive, en la cual se envían los archivos guardados con anterioridad.

En la figura 9 se muestra la carpeta del hospital-a posterior a la segunda corrida del pipeline, en la que se pueden ver los archivos parquet actualizados, así como la carpeta archive generada. En la figura 10, tenemos una copia de los archivos parquet, en la subcarpeta bronze/hosa/archive/2025/12/26, verificando que el pipeline se ejecuta de manera satisfactoria

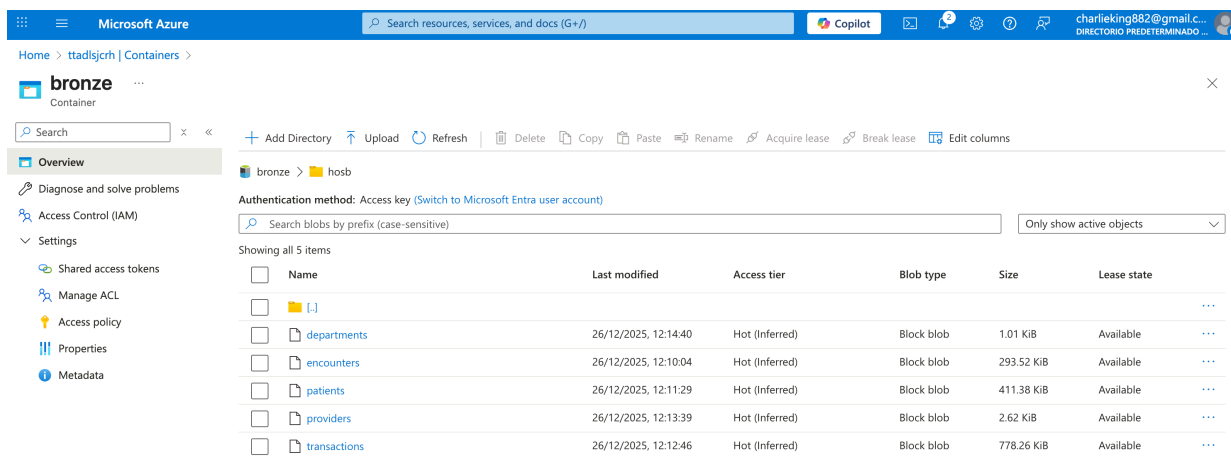


Figura 15.- Capa bronce dentro del data lake donde se observa la carga de los archivos parquet del hospital-B.

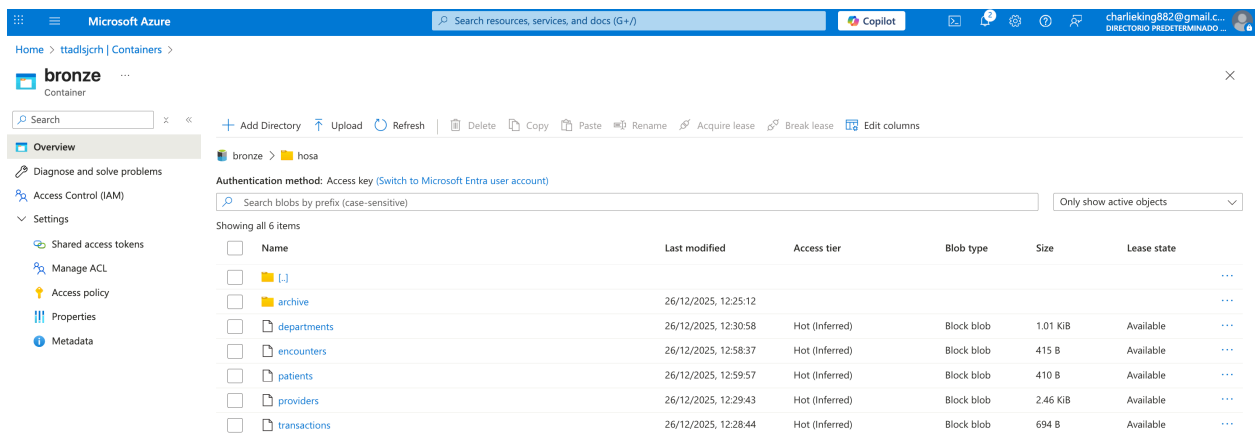


Figura 16.- Capa bronce dentro del data lake donde se observa la carga de los archivos parquet del hospital-A, así como la carpeta archive que almacena los archivos del histórico de datos.

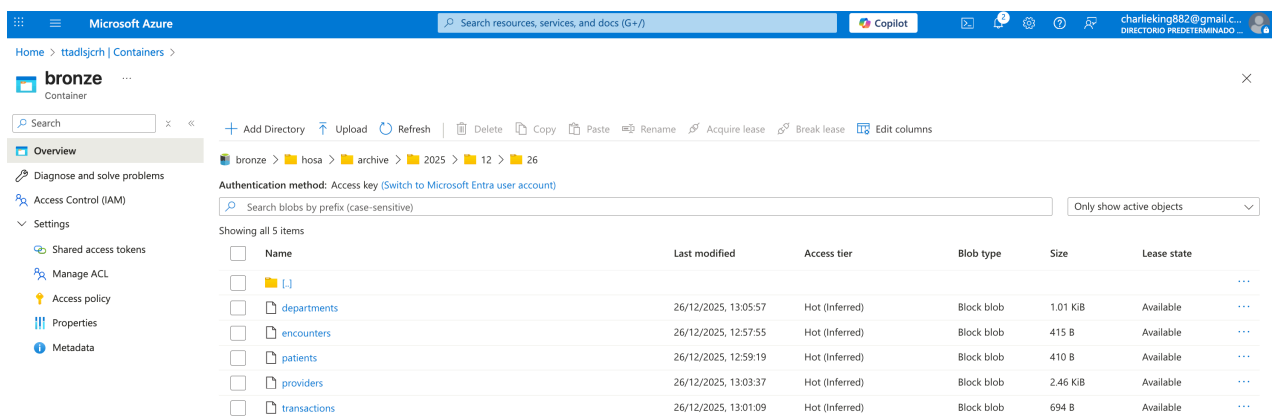


Figura 17.- Capa archive dentro del data lake donde se observa la carga de los archivos parquet del hospital-A, para el día 26/12/2025.

Se añaden 10 registros más a la tabla delta “audit.load_logs”, en la cual se puede observar que para el caso de las cargas Full se marca la totalidad de los filas copiadas mientras que para la carga Incremental solo añadiría las filas nuevas, para el ejercicio no se agregaron filas nuevas a los archivos por lo que las indica como cero(tabla 2).

id	data_source	tablename	numberofrowscopied	watermarkcolumnname	loaddate
1	hos-a	dbo.encounters	10000	ModifiedDate	2025-12-25T23:16:23.32
2	hos-a	dbo.patients	5000	ModifiedDate	2025-12-25T23:17:33.71
3	hos-a	dbo.transactions	10000	ModifiedDate	2025-12-25T23:18:52.49
4	hos-a	dbo.providers	25		2025-12-25T23:19:57.10
5	hos-a	dbo.departments	20		2025-12-25T23:20:55.32
6	hos-b	dbo.encounters	10000	ModifiedDate	2025-12-25T23:22:26.95
7	hos-b	dbo.patients	5000	Updated_Date	2025-12-25T23:23:51.95
8	hos-b	dbo.transactions	10000	ModifiedDate	2025-12-25T23:25:29.02
9	hos-b	dbo.providers	30		2025-12-25T23:26:01.43
10	hos-b	dbo.departments	20		2025-12-25T23:26:50.75

11	hos-a	dbo.encounters	0	ModifiedDate	2025-12-25T23:31:18.54
12	hos-a	dbo.patients	0	ModifiedDate	2025-12-25T23:32:54.75
13	hos-a	dbo.transactions	0	ModifiedDate	2025-12-25T23:34:19.34
14	hos-a	dbo.providers	25		2025-12-25T23:35:21.52
15	hos-a	dbo.departments	20		2025-12-25T23:36:28.75
16	hos-b	dbo.encounters	0	ModifiedDate	2025-12-25T23:38:06.05
17	hos-b	dbo.patients	0	Updated_Date	2025-12-25T23:39:26.08
18	hos-b	dbo.transactions	0	ModifiedDate	2025-12-25T23:40:59.95
19	hos-b	dbo.providers	30		2025-12-25T23:42:06.27
20	hos-b	dbo.departments	20		2025-12-25T23:43:20.60

Tabla 2.- Audit.load_logs table para la posterior a la primera corrida del pipeline.

Pipeline para la transformación de los datos de bronce a silver y de silver a gold

Una vez que tenemos los archivos en la capa bronce, procedemos a su lectura y transformaciones dentro de databricks. La figura 11 muestra los archivos cargados correctamente dentro del datalake.

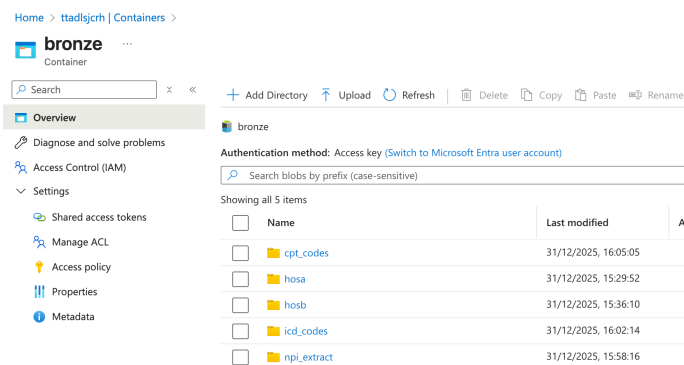


Figura 18.- Archivos en el contenedor bronce del datalake, se observa la correcta carga de los mismos.

A manera de ejemplo vamos a mostrar el caso de la dimensión patients, para mostrar la lectura, transformación y carga en azure databricks. Creamos una conexión entre databricks y el data lake con el servicio de Access-connector de Azure.

Creamos un notebook dentro de databrick con el siguiente código:

```
import pyspark.sql.functions as F

# Reading Hospital A patient data
df_hosa =
spark.read.parquet("abfss://bronze@ttadlsjcrh.dfs.core.windows.net/hosa/patients")

df_hosa.createOrReplaceTempView("patients_hosa")
```

```
# Reading Hospital B patient data
df_hosb =
spark.read.parquet("abfss://bronze@ttadlsjcrh.dfs.core.windows.net/hosb/patients")
df_hosb.createOrReplaceTempView("patients_hosb")
```

Dado que se tienen ciertas diferencias entre los schemas de la tabla patients entre el Hospital-A y Hospital-B, se generará un Common Data Model o CMD para unificar los schemas y tener en una sola tabla los registros de ambos hospitales, como sigue:

```
%sql
CREATE OR REPLACE TEMP VIEW cdm_patients AS
SELECT CONCAT(SRC_PatientID, '-', datasource) AS Patient_Key, *
FROM (
    SELECT
        PatientID AS SRC_PatientID,
        FirstName,
        LastName,
        MiddleName,
        SSN,
        PhoneNumber,
        Gender,
        DOB,
        Address,
        ModifiedDate,
        datasource
    FROM patients_hosa
    UNION ALL
    SELECT
        ID AS SRC_PatientID,
        F_Name AS FirstName,
        L_Name AS LastName,
        M_Name AS MiddleName,
        SSN,
        PhoneNumber,
        Gender,
        DOB,
        Address,
        Updated_Date AS ModifiedDate,
        datasource
    FROM patients_hosb
)
```

Se realiza un quality check para filtrar valores inválidos en función del id del paciente y su nombre, como sigue:

```
%sql
CREATE OR REPLACE TEMP VIEW quality_checks AS
SELECT
    Patient_Key,
    SRC_PatientID,
    FirstName,
    LastName,
    MiddleName,
    SSN,
    PhoneNumber,
    Gender,
    DOB,
    Address,
    ModifiedDate As SRC_ModifiedDate,
    datasource,
    CASE
        WHEN SRC_PatientID IS NULL OR dob IS NULL OR firstname IS NULL or
lower(firstname)='null' THEN TRUE
        ELSE FALSE
    END AS is_quarantined
FROM cdm_patients
```

Creamos la tabla en el unity catalog de databricks, para almacenar los datos.

```
%sql
CREATE TABLE IF NOT EXISTS silver.patients (
    Patient_Key STRING,
    SRC_PatientID STRING,
    FirstName STRING,
    LastName STRING,
    MiddleName STRING,
    SSN STRING,
    PhoneNumber STRING,
    Gender STRING,
    DOB DATE,
    Address STRING,
    SRC_ModifiedDate TIMESTAMP,
    datasource STRING,
    is_quarantined BOOLEAN,
    inserted_date TIMESTAMP,
    modified_date TIMESTAMP,
```

```
    is_current BOOLEAN  
)  
USING DELTA;
```

Para aplicar un Slowly Changing Dimension Tipo-2, el cual permite preservar un historial de los registros al añadir filas con los cambios o actualizaciones sin eliminar los anteriores. Para lo anterior, primero se identifican aquellos registros que sean los más recientes (`is_current = true`) y que hayan sufrido alguna modificación en alguna de sus columnas, se actualiza su valor en la columna `is_current` a `false`.

```
%sql  
-- Step 1: Mark existing records as historical (is_current = false) for patients  
-- that will be updated  
MERGE INTO silver.patients AS target  
USING quality_checks AS source  
ON target.Patient_Key = source.Patient_Key  
AND target.is_current = true  
WHEN MATCHED  
AND (  
    target.SRC_PatientID <> source.SRC_PatientID OR  
    target.FirstName <> source.FirstName OR  
    target.LastName <> source.LastName OR  
    target.MiddleName <> source.MiddleName OR  
    target.SSN <> source.SSN OR  
    target.PhoneNumber <> source.PhoneNumber OR  
    target.Gender <> source.Gender OR  
    target.DOB <> source.DOB OR  
    target.Address <> source.Address OR  
    target.SRC_ModifiedDate <> source.SRC_ModifiedDate OR  
    target.datasource <> source.datasource OR  
    target.is_quarantined <> source.is_quarantined  
)  
THEN UPDATE SET  
    target.is_current = false,  
    target.modified_date = current_timestamp()
```

Posteriormente se insertan los valores que nuevos o las actualizaciones de los cambios en los registros actuales.

```
%sql  
MERGE INTO silver.patients AS target  
USING quality_checks AS source  
ON target.Patient_Key = source.Patient_Key  
AND target.is_current = true
```

```

-- Step 2: Insert new and updated records into the Delta table, marking them as
current
WHEN NOT MATCHED
THEN INSERT (
    Patient_Key,
    SRC_PatientID,
    FirstName,
    LastName,
    MiddleName,
    SSN,
    PhoneNumber,
    Gender,
    DOB,
    Address,
    SRC_ModifiedDate,
    datasource,
    is_quarantined,
    inserted_date,
    modified_date,
    is_current
)
VALUES (
    source.Patient_Key,
    source.SRC_PatientID,
    source.FirstName,
    source.LastName,
    source.MiddleName,
    source.SSN,
    source.PhoneNumber,
    source.Gender,
    source.DOB,
    source.Address,
    source.SRC_ModifiedDate,
    source.datasource,
    source.is_quarantined,
    current_timestamp(), -- Set inserted_date to current timestamp
    current_timestamp(), -- Set modified_date to current timestamp
    true -- Mark as current
);

```

Por ejemplo si para el paciente con la Patient_Key 'HOSP1-000001-hos-a', se tiene una dirección inicial en la tabla target como sigue:

Patient_Key	SRC_PatientID	FirstName	LastName	Address	ModifiedDate	datasource	is_current
HOSP1-000001-hos-a	HOSP1-000001	Rick	Russo	Unit 0915 Box 7064, DPO AA 82777	05/25/2020	hos-a	true

Pensemos que en la tabla source(quality checks) se realizará una actualización en la dirección, tendríamos lo siguiente:

Patient_Key	SRC_PatientID	FirstName	LastName	sAddress	ModifiedDate	datasource	is_current
HOSP1-000001-hos-a	HOSP1-000001	Rick	Russo	160 Spear St, 15th Floor, San Francisco, CA 94105	12/15/2025	hos-a	true

El registro con la direccion anterior debe tener en su columna is_current = false y debe añadirse una nueva fila con la nueva dirección del paciente indicando la etiqueta is_current = true.

Al realizar los cambios la tabla target, en este caso silver.patients debe quedar como sigue:

Patient_Key	SRC_PatientID	FirstName	LastName	Address	ModifiedDate	datasource	is_current
HOSP1-000001-hos-a	HOSP1-000001	Rick	Russo	160 Spear St, 15th Floor, San Francisco, CA 94105	25/05/2020	hos-a	true
HOSP1-000001-hos-a	HOSP1-000001	Rick	Russo	Unit 0915 Box 7064, DPO AA 82777	25/05/2020	hos-a	false

Para la capa Gold, crearíamos la tabla en el unity catalog.

```
%sql
CREATE TABLE IF NOT EXISTS gold.dim_patient
(
    patient_key STRING,
    src_patientid STRING,
    firstname STRING,
    lastname STRING,
    middlename STRING,
    ssn STRING,
    phonenumber STRING,
    gender STRING,
    dob DATE,
    address STRING,
    datasource STRING
)
```

Insertaríamos los valores actuales, así como aquellos que se hayan ido a cuarentena.

```
%sql
insert into gold.dim_patient
select
    patient_key,
    src_patientid,
    firstname,
    lastname,
    middlename,
    ssn,
    phonenumber,
    gender,
    dob,
    address,
    datasource
from silver.patients
where is_current=true and is_quarantined=false
```

Por último los guardaríamos como una delta table en el data lake en la capa gold.

```
df_gol_dim_patients = spark.read.format("delta").table("gold.dim_patient")

# Define your ADLS Gen2 path
adls_path = "abfss://gold@ttadlsjcrh.dfs.core.windows.net/"

# Write the DataFrame as a Delta table
df_gol_dim_patients.write.format("delta").mode("overwrite").save(adls_path +
"/patients")
```

Lo anterior lo replicaríamos para cada tabla o dimension y fact tables.

Se crearon databricks notebooks activities para cada tabla, uno para el movimiento de bronce a silver y otro para moverlos de silver a gold como se muestra en la figura 18. Las tablas finales que confirman el data model se almacenan tanto en el data lake (figuras 19 y 20) como en el unity catalog en databricks (figura 21).

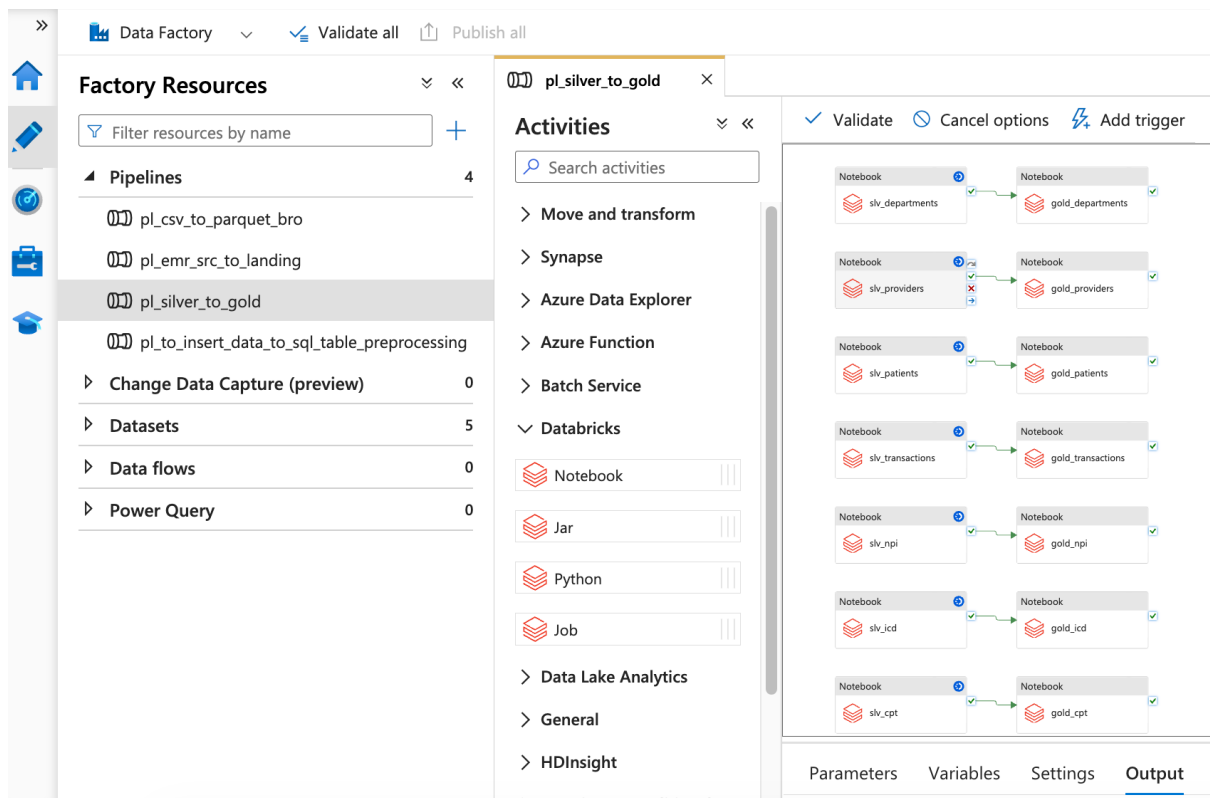


Figura 18.- Notebooks activities para las transformaciones de los datos de la capa bronce a silver y silver a gold.

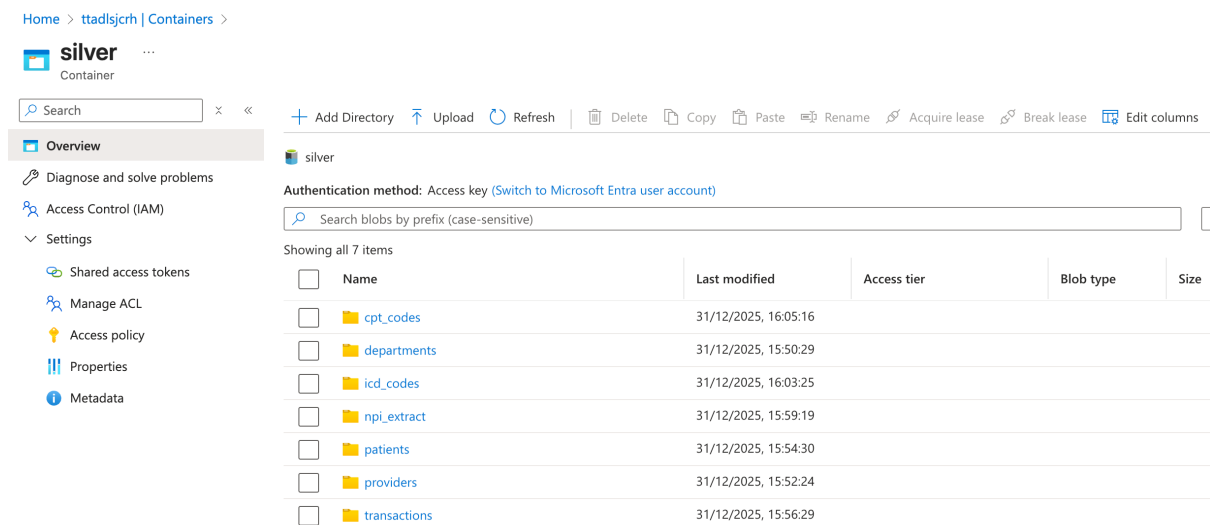


Figura 19.- Carpeta silver dentro del data lake en la cual se observa la correcta carga de los archivos delta.

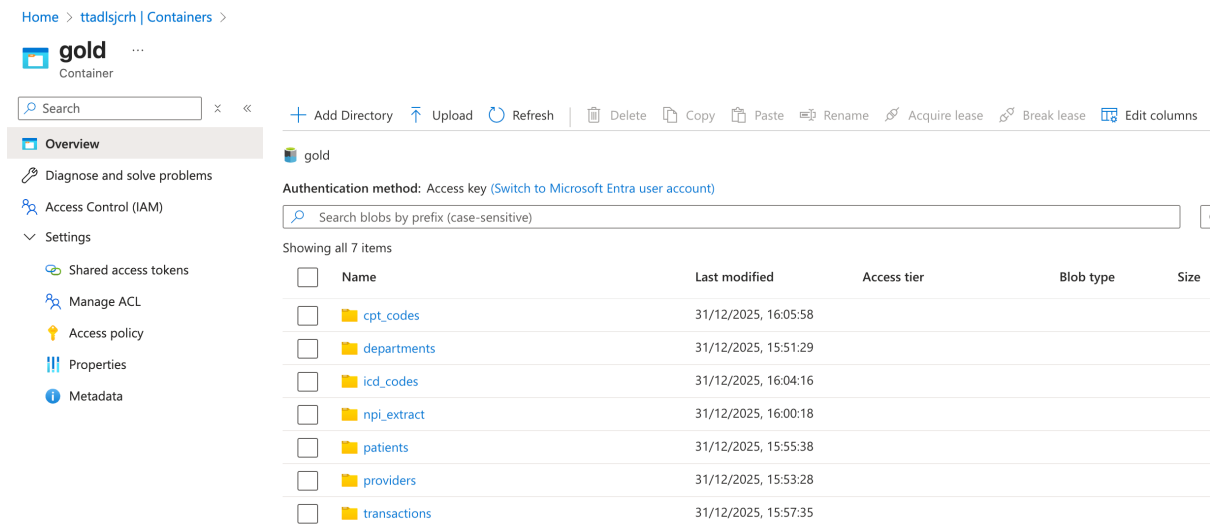


Figura 20.- Carpeta gold dentro del data lake en la cual se observa la correcta carga de los archivos delta.

Integración de pipelines - Pipeline end to end

Por último, crearíamos un pipeline que contenga los dos anteriores, el pipeline para traer los datos de las fuentes hacia bronce dentro del data lake y el pipeline para mover y transformar los datos hacia las capas silver y gold (figura 21).

Realizamos una corrida para verificar que no presenta algún error y confirmamos en el unity catalog de databricks la generación de las tablas (figura 22).

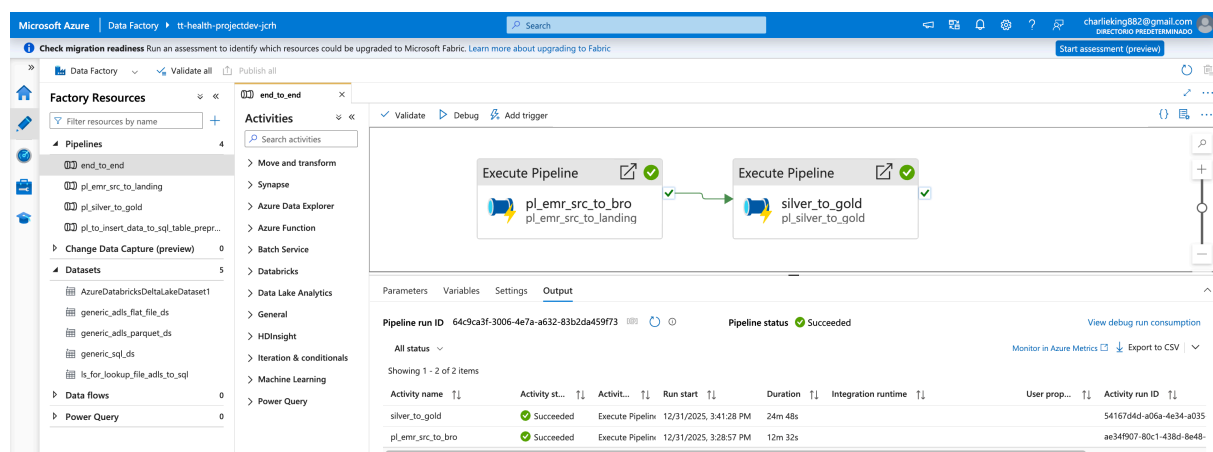


Figura 21.- Integración de pipelines de source a bronce y bronce a silver y gold en un pipeline end-to-end.

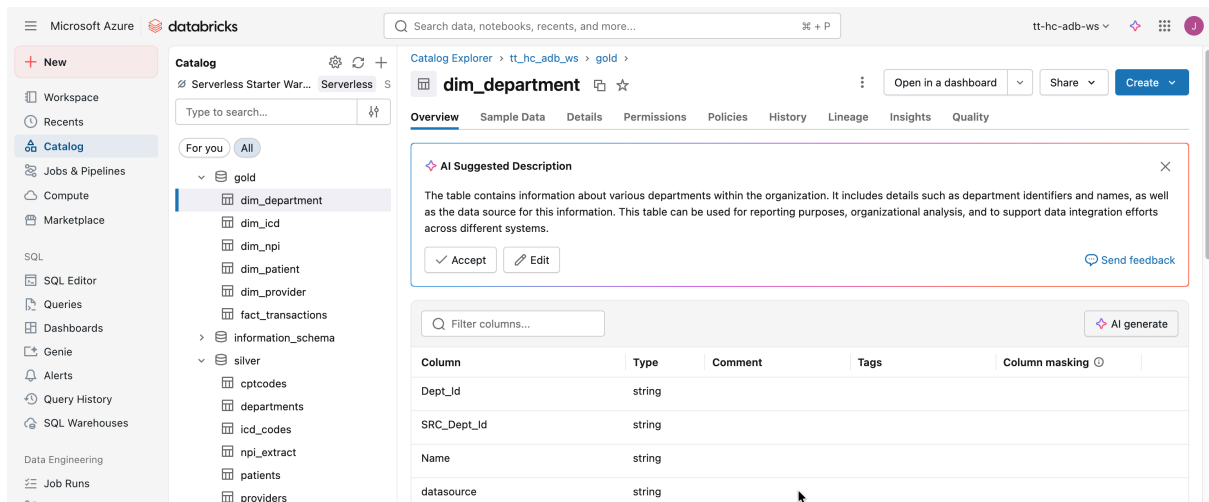


Figura 22.- Tablas delta dentro del unity catalog de databricks para las capas silver y gold.

Linked services y datasets

Para el proyecto se utilizaron 5 linked services y 5 datasets para establecer las conexiones entre los distintos servicios y archivos. Una conexión hacia AzureDatabricksDeltaLake para cargar los logs en la tabla audit.load_logs, el AzureDataLakeStorage1 para las conexiones hacia el data lake, una conexión hosa_sql_ls para los bases de datos en Azure SQL, una tt-hc-adb-ls para las conexiones hacia databricks y una ultima tt-hc-kc-ls para la conexión hacia el key-vault.

En relación a los datasets se crearon multiples para formatos de datos como csv, parquet, para las bases de datos en Azure SQL y una para leer el json en la actividad Lookup (figura 23).

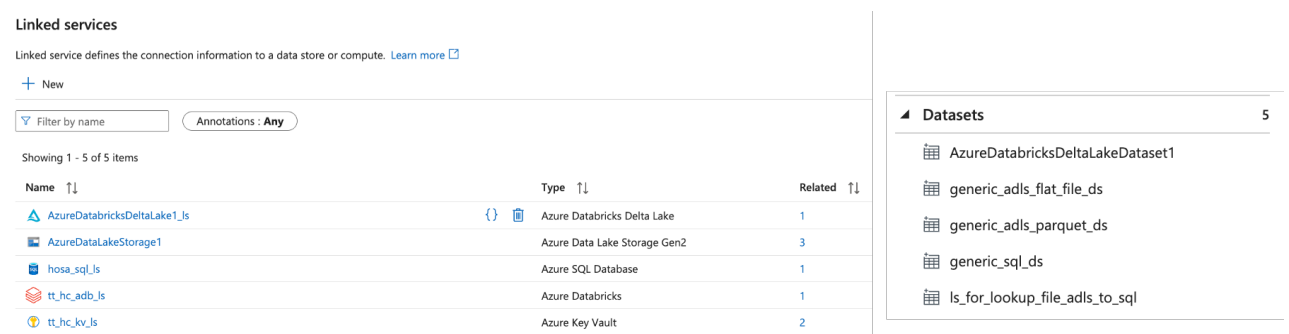


Figura 23.- Linked services (izquierda) y datasets (derecha) utilizados para el proyecto healthcare.